

Projeto STOCK - Gestão de Estoque para Laboratórios

Status do Projeto

Fase: Planejamento / Definição de Arquitetura

Data de início: 13/07/2026

Última atualização: 16/07/2026

Visão Geral

Nome do projeto: Stock

Tipo: Sistema de Gestão de Estoque para Laboratórios

Objetivo: Automatizar e centralizar o controle de materiais em laboratórios de pesquisa/ensino

Conceito e Diferencial

O que é o projeto?

Sistema completo para gestão de estoque de laboratórios, controlando entrada/saída de materiais, vinculando estoque a projetos e pesquisadores, com armazenamento seguro de documentos (pedidos, relatórios, etc.).

Por que é diferente?

- ****Foco em laboratórios**** - Não é um genérico de estoque, é específico para o contexto de pesquisa/ensino
- ****Integração com projetos**** - Cada pedido está atrelado a um projeto e pesquisador
- ****Hierarquia organizacional**** - Respeita a estrutura: Unidade !' Laboratório !' Pesquisador
- ****Armazenamento de documentos**** - Guarda pedidos, relatórios e comprovantes
- ****Base sólida para expansão**** - API REST preparada para novas funcionalidades

Hierarquia do Sistema

[illegible]

[illegible]

Estrutura de Pastas (Proposta)

stock/	
% % % backend/	# API Spring Boot
% % % % src/main/java/	
% % % % com.stock/	
% % % % controller/	# Endpoints REST
% % % % service/	# Lógica de negócio
% % % % repository/	# Acesso a dados
% % % % model/	# Entidades
% % % % dto/	# Data Transfer Objects
% % % % config/	# Configurações
% % % % pom.xml	
%	
% % % frontend/	# Vue.js
% % % % src/	
% % % % components/	# Componentes reutilizáveis
% % % % views/	# Páginas
% % % % services/	# Chamadas à API
% % % % store/	# Estado global
% % % % router/	# Rotas
% % % % package.json	
%	
% % % docs/	# Documentação
% % % % CONTINUIDADE.md	# Este arquivo
%	
% % % docker-compose.yml	# Orquestração (opcional)

Modelo de Dados (Entidades Principais)

Unidade (Tenant)

- id
- nome
- codigo
- ativo

Laboratório

- id
- unidade_id (FK)
- nome
- descricao
- responsavel
- ativo

Estudante/Pesquisador

- id
- laboratorio_id (FK)
- nome
- email
- matricula
- tipo (ESTUDANTE, PESQUISADOR, PROFESSOR)
- ativo

Produto/Item

- id
 - laboratorio_id (FK)
 - nome
 - descricao
 - codigo_referencia
 - quantidade_atual
 - quantidade_minima !• Alerta quando atingir
 - unidade_medida
 - localizacao_fisica
 - ativo
- // Campos de risco e perecibilidade
- risco (enum: NENHUM, BAIXO, MEDIO, ALTO)
 - tipo_risco (enum: NENHUM, INFLAMAVEL, RADIOATIVO, TOXICO, CORROSIVO, BIOLOGICO)
 - descricao_risco (texto livre para detalhes especificos, ex: "Material radioativo Classe 3")
 - classe_risco (boolean)
 - dias_validade (inteiro, opcional - para alertas de validade)
 - tipo_perecivel (enum: NENHUM, VEGETAL, ANIMAL, MICROBIANO, QUIMICO)
 - condicoes_armazenamento (texto livre, ex: "Armazenar em freezer -20°C")

Pedido

- id
- estudante_id (FK)
- laboratorio_id (FK)
- data_solicitacao
- status (PENDENTE, APROVADO, REJEITADO, ENTREGUE)
- observacao
- arquivo_documento (URL/path do documento)

ItemPedido (itens do pedido)

- id
- pedido_id (FK)
- produto_id (FK)
- quantidade_solicitada
- quantidade_aprovada

Projeto

- id
- laboratorio_id (FK)

- nome
 - descricao
 - data_inicio
 - data_fim
 - responsavel
 - ativo
-

Funcionalidades

MVP (Mínimo Viável)

Gestão de Estoque

- ☐ Cadastro de produtos com quantidade mínima
- ☐ Entrada de material (compra, doação, transferência)
- ☐ Saída de material (pedido aprovado)
- ☐ Alertas automáticos de estoque baixo
- ☐ Consulta de estoque por laboratório
- ☐ Classificação de risco dos produtos (Nenhum/Baixo/Médio/Alto)
- ☐ Tipo de risco (Inflamável/Radioativo/Tóxico/Corrosivo/Biológico)
- ☐ Cadastro de produtos perecíveis com controle de validade
- ☐ Alertas de validade próxima

Participantes

- ☐ Cadastro de unidades (tenants)
- ☐ Cadastro de laboratórios
- ☐ Cadastro de estudantes/pesquisadores
- ☐ Vinculação pesquisador !" laboratório
- ☐ Vinculação pesquisador !" projeto

Pedidos

- ☐ Solicitação de material pelo pesquisador
- ☐ Aprovação/rejeição pelo responsável
- ☐ Baixa automática no estoque
- ☐ Histórico de pedidos

Armazenamento de Documentos

- ☐ Upload de pedidos (PDF, imagem)
- ☐ Upload de relatórios
- ☐ Download de documentos
- ☐ Organização por laboratório/pedido

Funcionalidades Futuras

- ☐ Relatórios gerenciais
- ☐ Exportação (PDF, Excel)

- [] Notificações por email
 - [] Dashboard com gráficos
 - [] Integração com sistemas externos
 - [] Controle de validade de produtos
 - [] Código de barras/QR Code
 - [] Relatório de riscos por laboratório
 - [] Histórico de manuseio de materiais perigosos
-

Controle de Risco e Perecibilidade

Níveis de Risco

Nível	Descrição	Exemplos
-------	-----------	----------

-----	-----	-----
-------	-------	-------

NENHUM	Sem risco identificado	Materiais de escritório, vidrarias comuns
--------	------------------------	---

BAIXO	Risco mínimo, manuseio padrão	Solventes diluídos, materiais biológicos inativos
-------	-------------------------------	---

MÉDIO	Requer cuidados específicos	Inflamáveis concentrados, materiais biológicos ativos
-------	-----------------------------	---

ALTO	Requer protocolo especial	Radioativos, materiais altamente tóxicos, agentes patogênicos
------	---------------------------	---

Tipos de Risco

Tipo	Descrição
------	-----------

-----	-----
-------	-------

INFLAMAVEL	Materiais que pegam fogo facilmente
------------	-------------------------------------

RADIOATIVO	Emissores de radiação (requer blindagem)
------------	--

TOXICO	Tóxicos para contato/ingestão/inalação
--------	--

CORROSIVO	Danificam materiais e tecidos
-----------	-------------------------------

BIOLOGICO	Agentes biológicos (bactérias, vírus, fungos)
-----------	---

Tipos de Perecibilidade

Tipo	Descrição
------	-----------

-----	-----
-------	-------

VEGETAL	Plantas, extratos vegetais, culturas de tecidos
---------	---

ANIMAL	Tecidos animais, soro, antígenos
--------	----------------------------------

MICROBIANO	Bactérias, vírus, fungos, leveduras
------------	-------------------------------------

QUIMICO	Compostos químicos instáveis
---------	------------------------------

Regras de Negócio para Risco/Perecibilidade

1. ****Produtos de risco ALTO**** exigem confirmação extra antes de liberar pedido
 2. ****Produtos perecíveis**** têm alerta automático de validade (configurável)
 3. ****Produtos radioativos**** podem exigir campos adicionais (atividade, data-calibração)
 4. ****Relatório de risco**** disponível para gestores (por lab, por tipo)
-

Endpoints da API (Exemplos)

Unidades

GET	/api/v1/unidades	- Listar unidades
POST	/api/v1/unidades	- Criar unidade
GET	/api/v1/unidades/{id}	- Buscar unidade
PUT	/api/v1/unidades/{id}	- Atualizar unidade
DELETE	/api/v1/unidades/{id}	- Remover unidade

Laboratórios

GET	/api/v1/laboratorios	- Listar laboratórios
POST	/api/v1/laboratorios	- Criar laboratório
GET	/api/v1/laboratorios/{id}	- Buscar laboratório
PUT	/api/v1/laboratorios/{id}	- Atualizar laboratório
GET	/api/v1/unidades/{id}/laboratorios	- Listar labs de uma unidade

Produtos

GET	/api/v1/produtos	- Listar produtos
POST	/api/v1/produtos	- Cadastrar produto
PUT	/api/v1/produtos/{id}	- Atualizar produto
GET	/api/v1/laboratorios/{id}/produtos	- Produtos do laboratório
GET	/api/v1/produtos/estoque-baixo	- Listar com estoque baixo
GET	/api/v1/produtos/risco/{nivel}	- Filtrar por nível de risco
GET	/api/v1/produtos/pereciveis	- Listar perecíveis
GET	/api/v1/produtos/validade-proxima	- Alertas de validade

Pedidos

GET	/api/v1/pedidos	- Listar pedidos
POST	/api/v1/pedidos	- Criar pedido
PUT	/api/v1/pedidos/{id}/aprovar	- Aprovar pedido
PUT	/api/v1/pedidos/{id}/rejeitar	- Rejeitar pedido
PUT	/api/v1/pedidos/{id}/entregar	- Marcar como entregue

Documentos

POST	/api/v1/pedidos/{id}/documentos	- Upload documento
GET	/api/v1/pedidos/{id}/documentos	- Listar documentos

Decisões Tomadas

| Data | Decisão | Motivo |

|-----|-----|-----|

| 13/07/2026 | Criar projeto | Ideia inicial do app de estoque |

| 13/07/2026 | Java Spring Boot | Base sólida, escalável, preparado para expansão |

| 13/07/2026 | Vue.js no frontend | Moderno, reativo, fácil integração com API |

| 13/07/2026 | PostgreSQL | Confiável, suporte a JSON, open source |

| 13/07/2026 | Modelo multi-tenant | Unidade = Tenant, permite múltiplas instituições |

| 16/07/2026 | Classificação de risco | Laboratórios lidam com materiais perigosos (radioativos, inflamáveis, biológicos) |

| 16/07/2026 | Controle de perecibilidade | Produtos como plantas, bactérias e frutos precisam de controle de validade |

| 16/07/2026 | Ambientes via branches Git primeiro | Simplicidade inicial, banco único enquanto o core é construído |

| 16/07/2026 | Bancos separados por ambiente como evolução | Após base do projeto estar pronta |

| 16/07/2026 | Nuvem/volumes adiados | Prioridade é ter o projeto base funcionando antes de pensar em infraestrutura de produção |

| 16/07/2026 | Spring Boot 4.1.0 | Versão mais recente, com melhorias de performance e segurança |

Em Andamento

- [x] Definição da ideia principal (13/07/2026)
- [x] Definição dos 3 pilares (13/07/2026)
- [x] Escolha das tecnologias (13/07/2026)
- [x] Criação da estrutura base Spring Boot (16/07/2026)
- [x] Configurar application.properties (16/07/2026)
- [x] Configurar SecurityConfig para H2 Console (16/07/2026)
- [x] Criar entidade Unidade (17/07/2026)
- [x] Criar entidade Laboratório (17/07/2026)
- [x] Criar entidade Estudante/Pesquisador (17/07/2026)
- [x] Criar entidade Produto (17/07/2026)
- [x] Criar entidade Pedido (17/07/2026)
- [x] Criar entidade ItemPedido (17/07/2026)

- ☒ Criar entidade Projeto (17/07/2026)
 - ☐ Prototipação das telas
 - ☐ Criação do banco de dados
 - ☐ Desenvolvimento da API
-

Concluído

- ☒ Criação da pasta do projeto (13/07/2026)
 - ☒ Criação do arquivo de continuidade (13/07/2026)
 - ☒ Definição da arquitetura (13/07/2026)
 - ☒ Definição do modelo de dados (13/07/2026)
 - ☒ Definição de controle de risco e peregibilidade (16/07/2026)
 - ☒ Definição de estratégia de ambientes (16/07/2026)
 - ☒ Criação da estrutura base Spring Boot (16/07/2026)
 - ☒ Configuração do application.properties (16/07/2026)
 - ☒ Configuração do SecurityConfig para H2 Console (16/07/2026)
 - ☒ Criação da entidade Unidade (17/07/2026)
 - ☒ Criação da entidade Laboratório (17/07/2026)
 - ☒ Criação da entidade Estudante/Pesquisador (17/07/2026)
 - ☒ Criação da entidade Produto (17/07/2026)
 - ☒ Criação da entidade Pedido (17/07/2026)
 - ☒ Criação da entidade ItemPedido (17/07/2026)
 - ☒ Criação da entidade Projeto (17/07/2026)
-

Bloqueios e Pendências

Item	Descrição	Responsável	Status
-----	-----	-----	-----
-	Nenhum no momento	-	-

Próximos Passos

Curto Prazo

1. ****Criar repositório Git**** - Versionar o código (ADIADO)
 2. ****Configurar Spring Boot**** - Projeto base com dependências (EM ANDAMENTO)
 3. ****Criar banco de dados**** - Scripts SQL das tabelas
 4. ****Desenvolver CRUDs básicos**** - Unidades, Labs, Produtos
-

Guia: Criando o Projeto Spring Boot (Spring Initializr)

Passo 1: Acessar o Spring Initializr

- Acesse: <https://start.spring.io/>
- Alternativa: <https://spring.io/initializr>

Passo 2: Configurar o Projeto

Preencha os campos:

- **Project:** Maven
- **Language:** Java
- **Spring Boot:** 4.1.0
- **Group:** `com.stock`
- **Artifact:** `stock-backend`
- **Name:** `stock-backend`
- **Description:** Gestão de Estoque para Laboratórios
- **Package name:** `com.stock`
- **Packaging:** Jar
- **Java:** 17

Passo 3: Selecionar Dependências

Adicione as seguintes dependências:

Core:

- Spring Web (para APIs REST)
- Spring Data JPA (para acesso ao banco)
- Validation (para validação de dados)

Banco de Dados:

- PostgreSQL Driver
- H2 Database (para testes locais)

Segurança:

- Spring Security
- OAuth2 Resource Server (para JWT futuro)

Outros:

- Lombok (para reduzir código boilerplate)
- MapStruct (opcional, para mapeamento de DTOs)
- SpringDoc OpenAPI (para documentação da API)

Testes:

- Spring Boot Test
- Testcontainers (opcional, para testes de integração)

Passo 4: Gerar e Baixar

- Clique em "GENERATE"

- Salve o arquivo `stock-backend.zip`
- Descompacte na pasta `C:\Users\07548262523\Documents\stock\backend\`

Passo 5: Estrutura Criada

O Spring Initializr criará:

```
backend/
% % % src/
%   % % % main/
%   %   % % % java/
%   %   %   % % % com/stock/
%   %   %   %   % % % StockBackendApplication.java
%   %   %   % % % resources/
%   %   %   %   % % % application.properties
%   %   %   %   % % % static/
%   %   %   %   % % % templates/
%   % % % test/
%       % % % java/
%       %   % % % com/stock/
%       %   %   % % % StockBackendApplicationTests.java
% % % pom.xml
% % % .mvn/
% % % % wrapper/
```

Passo 6: Configuração Inicial

1. ****Abrir no IDE**** (IntelliJ, Eclipse, VS Code)
2. ****Importar projeto Maven****
3. ****Executar teste básico**** para verificar se compila:

```
cd backend
mvn clean compile
```

Passo 7: Verificar Dependências

Execute no terminal:

```
mvn dependency:tree
```

Verifique se todas as dependências foram baixadas corretamente.

Passo 8: Configurar SecurityConfig (para H2 Console)

Crie o arquivo backend/stock-backend/src/main/java/com/stock/config/SecurityConfig.java:

```
package com.stock.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.web.header.writers.ReferrerPolicyHeaderWriter;
```

```

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .csrf(csrf -> csrf.disable())
            .headers(headers -> headers
                .referrerPolicy(referrer ->
                    referrer.policy(ReferrerPolicyHeaderWriter.ReferrerPolicy.NO_REFERRER)
                        .frameOptions(frame -> frame.disable())
                )
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/h2-console/**").permitAll()
                .anyRequest().permitAll()
            )
            .formLogin(form -> form.disable())
            .httpBasic(basic -> basic.disable());

        return http.build();
    }
}

```

Passo 9: Configurar application.properties

```

# Servidor
server.port=8080

# Banco de Dados (desenvolvimento com H2)
spring.datasource.url=jdbc:h2:mem:stockdb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.h2.console.enabled=true

# JPA
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

# PostgreSQL (produção) - descomente quando necessário
# spring.datasource.url=jdbc:postgresql://localhost:5432/stock
# spring.datasource.driverClassName=org.postgresql.Driver
# spring.datasource.username=postgres
# spring.datasource.password=sua_senha
# spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect

**Arquivo:** backend/stock-backend/src/main/resources/application.properties

```

Passo 10:

Tester H2 Console

1. Reinicie a aplicação: `mvn spring-boot:run``
2. Acesse: `http://localhost:8080/h2-console`
3. Login:

- **JDBC URL:** `jdbc:h2:mem:stockdb`
 - **User Name:** `sa`
 - **Password:** (vazio)
4. Clique em "Connect"
 5. Deve aparecer a tela do console H2 (vazia, sem tabelas ainda)

Passo 11: Próximos Passos no Código

1. Criar primeira entidade (Unidade)
2. Criar Repository
3. Criar Service
4. Criar Controller
5. Testar CRUD básico

Passo 9: Testar a Aplicação

```
mvn spring-boot:run
```

Acesse: <http://localhost:8080>

Passo 10: Próximos Passos no Código

1. Criar primeira entidade (Unidade)
 2. Criar Repository
 3. Criar Service
 4. Criar Controller
 5. Testar CRUD básico
-

Próximo Passo: Configurar application.properties

Ação Necessária

Edite o arquivo `backend/stock-backend/src/main/resources/application.properties` e adicione as seguintes configurações:

```
# Servidor
server.port=8080

# Banco de Dados (desenvolvimento com H2)
spring.datasource.url=jdbc:h2:mem:stockdb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.h2.console.enabled=true

# JPA
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

Por que essas configurações?

- **server.port=8080:** Porta padrão para APIs REST
- **H2 Database:** Banco de dados em memória para desenvolvimento (não precisa instalar nada)
- **spring.h2.console.enabled=true:** Permite acessar o console do banco via navegador
- **spring.jpa.hibernate.ddl-auto=update:** Cria/atualiza tabelas automaticamente
- **spring.jpa.show-sql=true:** Mostra SQL executado no console (útil para debug)

Verificação

Após salvar, rode novamente:

```
cd backend/stock-backend
mvn spring-boot:run
```

Acesse: <http://localhost:8080>

Console H2: <http://localhost:8080/h2-console>

Checklist de Acompanhamento: docs/checklist-estrutura-base.md

Guia: Criando a

Primeira Entidade - Unidade

Passo 1: Criar estrutura de pastas

No diretório `backend/stock-backend/src/main/java/com/stock/`, crie as seguintes pastas:

% % %	config/	(já existe - SecurityConfig)	com/stock/
% % %	model/	(criar)	
% % %	repository/	(criar)	
% % %	service/	(criar)	
% % %	controller/	(criar)	

Passo 2: Criar Model - Unidade.java

Crie o arquivo `model/Unidade.java`:

```
package com.stock.model;

import jakarta.persistence.*;
import lombok.*;

@Entity
@Table(name = "unidades")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
```

```

@Builder
public class Unidade {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String nome;

    @Column(nullable = false, unique = true)
    private String codigo;

    @Column(nullable = false)
    private Boolean ativo = true;
}

```

Passo 3: Criar Repository - UnidadeRepository.java

Crie o arquivo repository/UnidadeRepository.java:

```

package com.stock.repository;

import com.stock.model.Unidade;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface UnidadeRepository extends JpaRepository<Unidade, Long> {
}

```

Passo 4: Criar Service - UnidadeService.java

Crie o arquivo service/UnidadeService.java:

```

package com.stock.service;

import com.stock.model.Unidade;
import com.stock.repository.UnidadeRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
@RequiredArgsConstructor
public class UnidadeService {

    private final UnidadeRepository unidadeRepository;

    public List<Unidade> listarTodos() {
        return unidadeRepository.findAll();
    }

    public Unidade buscarPorId(Long id) {
        return unidadeRepository.findById(id)

```

```

        .orElseThrow(() -> new RuntimeException("Unidade não encontrada com id:
" + id));

    public Unidade salvar(Unidade unidade) {
        return unidadeRepository.save(unidade);
    }

    public Unidade atualizar(Long id, Unidade unidadeAtualizada) {
        Unidade unidade = buscarPorId(id);
        unidade.setNome(unidadeAtualizada.getNome());
        unidade.setCodigo(unidadeAtualizada.getCodigo());
        unidade.setAtivo(unidadeAtualizada.getAtivo());
        return unidadeRepository.save(unidade);
    }

    public void deletar(Long id) {
        unidadeRepository.deleteById(id);
    }
}

```

Passo 5: Criar Controller - UnidadeController.java

Crie o arquivo controller/UnidadeController.java:

```

package com.stock.controller;

import com.stock.model.Unidade;
import com.stock.service.UnidadeService;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/v1/unidades")
@RequiredArgsConstructor
public class UnidadeController {

    private final UnidadeService unidadeService;

    @GetMapping
    public ResponseEntity<List<Unidade>> listarTodos() {
        return ResponseEntity.ok(unidadeService.listarTodos());
    }

    @GetMapping("/{id}")
    public ResponseEntity<Unidade> buscarPorId(@PathVariable Long id) {
        return ResponseEntity.ok(unidadeService.buscarPorId(id));
    }

    @PostMapping
    public ResponseEntity<Unidade> salvar(@RequestBody Unidade unidade) {
        Unidade novaUnidade = unidadeService.salvar(unidade);
        return ResponseEntity.status(HttpStatus.CREATED).body(novaUnidade);
    }
}

```



```

    @PutMapping("/{id}")
    public ResponseEntity<Unidade> atualizar(@PathVariable Long id, @RequestBody Unidade
    unidade) {
        return ResponseEntity.ok(unidadeService.atualizar(id, unidade));
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deletar(@PathVariable Long id) {
        unidadeService.deletar(id);
        return ResponseEntity.noContent().build();
    }
}

```

Passo 6: Testar CRUD

1. Reinicie a aplicação: `mvn spring-boot:run`
2. Acesse o H2 Console e verifique se a tabela `unidades` foi criada
3. Teste os endpoints:

```

# Criar unidade
curl -X POST http://localhost:8080/api/v1/unidades \
  -H "Content-Type: application/json" \
  -d '{"nome": "Instituto de Biologia", "codigo": "IB", "ativo": true}'

# Listar unidades
curl http://localhost:8080/api/v1/unidades

# Buscar por ID
curl http://localhost:8080/api/v1/unidades/1

# Atualizar
curl -X PUT http://localhost:8080/api/v1/unidades/1 \
  -H "Content-Type: application/json" \
  -d '{"nome": "Instituto de Biologia Atualizado", "codigo": "IB", "ativo": true}'

# Deletar
curl -X DELETE http://localhost:8080/api/v1/unidades/1

```

Estrutura Final

```

backend/stock-backend/src/main/java/com/stock/
% % % StockBackendApplication.java
% % % config/
%   % % % SecurityConfig.java
% % % model/
%   % % % Unidade.java
% % % repository/
%   % % % UnidadeRepository.java
% % % service/
%   % % % UnidadeService.java
% % % controller/
%   % % % UnidadeController.java

```

IMPORTANTE: Este guia é para você seguir passo a passo. Eu (Mimo) estou aqui apenas para orientar, documentar e fazer checkups. Você mesmo criará o código!

Papéis Definidos

Você (Desenvolvedor/Estudante)

- ****Responsável por:**** Criar todo o código, métodos, classes, funções
- ****Objetivo:**** Treinar e entender o sistema todo
- ****Ação:**** Seguir os guias e checklists, implementar cada passo

Mimo (Orientador)

- ****Responsável por:**** Documentação, orientação, checkups
- ****Objetivo:**** Guiar, explicar conceitos, verificar progresso
- ****Ação:**** Fornecer guias, responder dúvidas, validar etapas

Regra de Ouro

> **Você cria o código. Eu oriento. Juntos garantimos a qualidade.**

Médio Prazo

5. ****Implementar lógica de pedidos**** - Fluxo de aprovação
6. ****Alertas de estoque baixo**** - Notificação automática
7. ****Upload de documentos**** - Armazenamento de arquivos
8. ****Frontend**** - Telas principais com Vue.js

Longo Prazo

9. ****Relatórios**** - Dashboard e exportações
10. ****Notificações**** - Email, push
11. ****Integrações**** - Sistemas externos

Recursos e Links

- [] Repositório Git: [CRIAR]
 - [] Protótipo/Figma: [CRIAR]
 - [] Documentação da API (Swagger): [CRIAR]
 - [] Banco de dados: [CRIAR SCRIPTS]
-

Contatos

| Nome | Papel | Contato |

|-----|-----|-----|

| [PREENCHER] | Desenvolvedor | [PREENCHER] |

Notas e Observações

Stack Escolhida

- **Backend:** Java Spring Boot (robusto, escalável, ecossistema maduro)
- **Frontend:** Vue.js (moderno, reativo, fácil de aprender)
- **Banco:** PostgreSQL (confiável, open source, suporte a JSON)
- **API:** REST (padrão de mercado, fácil integração)

Conceitos Importantes

- **Multi-tenant:** Cada Unidade é um tenant separado
- **Estoque mínimo:** Produto define quantidade mínima para alerta
- **Fluxo de pedido:** Pesquisador solicita !' Responsável aprova !' Estoque baixa
- **Armazenamento:** Documentos ficam vinculados aos pedidos
- **Risco:** Classificação Nenhum/Baixo/Médio/Alto com tipo específico (radioativo, inflamável, etc)
- **Perecibilidade:** Controle de validade para produtos biológicos, vegetais, animais

Para Continuar o Projeto

1. Ler este arquivo primeiro
 2. Seguir a ordem dos "Próximos Passos"
 3. Atualizar este arquivo com decisões e progresso
 4. Manter o checklist atualizado
-

Estratégia de Ambientes

Fase 1 (Atual) - Separação via branches Git

- **Branch `main`:** Produção/Release (código estável)
- **Branch `develop` ou `alpha`:** Desenvolvimento/Testes
- **Banco de dados único compartilhado** enquanto o projeto está em construção do core
- Simplicidade inicial, sem custo de infraestrutura adicional

Fase 2 (Futura) - Bancos separados por ambiente

- Após o projeto base estar consolidado

- **Alpha**: Banco de dados separado para testes e validação
- **Produção**: Banco de dados separado para uso final
- Permite testes mais realistas sem afetar dados de produção

Fase 3 (Futura, não priorizada) - Nuvem/Infraestrutura

- Avaliar volumes e necessidades de infraestrutura
- Considerar serviços gerenciados (banco, armazenamento, etc.)
- **Decisão adiada** até o projeto base estar funcional
- Prioridade é ter o sistema rodando antes de otimizar infraestrutura

IMPORTANTE: Este arquivo é o ponto de continuidade do projeto. Qualquer pessoa ou IA que pegar este projeto deve ler este arquivo primeiro para entender o contexto e continuar de onde paramos.